
Rethinking Graph Transformers with Spectral Attention

Devin Kreuzer *

McGill University, Mila
Montreal, Canada
devin.kreuzer@mail.mcgill.ca

Dominique Beaini *

Valence Discovery
Montreal, Canada
dominique@valencediscovery.com

William L. Hamilton

McGill University, Mila
Montreal, Canada
wlh@cs.mcgill.ca

Vincent Létourneau

University of Ottawa
Ottawa, Canada
vletour2@uottawa.ca

Prudencio Tossou

Valence Discovery
Montreal, Canada
prudencio@valencediscovery.com

Abstract

In recent years, the Transformer architecture has proven to be very successful in sequence processing, but its application to other data structures, such as graphs, has remained limited due to the difficulty of properly defining positions. Here, we present the *Spectral Attention Network* (SAN), which uses a learned positional encoding (LPE) that can take advantage of the full Laplacian spectrum to learn the position of each node in a given graph. This LPE is then added to the node features of the graph and passed to a fully-connected Transformer. By leveraging the full spectrum of the Laplacian, our model is theoretically powerful in distinguishing graphs, and can better detect similar sub-structures from their resonance. Further, by fully connecting the graph, the Transformer does not suffer from over-squashing, an information bottleneck of most GNNs, and enables better modeling of physical phenomena such as heat transfer and electric interaction. When tested empirically on a set of 4 standard datasets, our model performs on par or better than state-of-the-art GNNs, and outperforms any attention-based model by a wide margin, becoming the first fully-connected architecture to perform well on graph benchmarks.

1 Introduction

The prevailing strategy for graph neural networks (GNNs) has been to directly encode graph structure through a sparse message-passing process [17, 19]. In this approach, vector messages are iteratively passed between nodes that are connected in the graph. Multiple instantiations of this message-passing paradigm have been proposed, differing in the architectural details of the message-passing apparatus (see [19] for a review).

However, there is a growing recognition that the message-passing paradigm has inherent limitations. The expressive power of message passing appears inexorably bounded by the Weisfeiler-Lehman isomorphism hierarchy [29, 30, 39]. Message-passing GNNs are known to suffer from pathologies, such

*Equal contribution.

as *oversmoothing*, due to their repeated aggregation of local information [19], and *over-squashing*, due to the exponential blow-up in computation paths as the model depth increases [1].

As a result, there is a growing interest in deep learning techniques that encode graph structure as a *soft inductive bias*, rather than as a hard-coded aspect of message passing [14, 24]. A central issue with message-passing paradigm is that input graph structure is encoded by restricting the structure of the model’s computation graph, inherently limiting its flexibility. This reminds us of how early recurrent neural networks (RNNs) encoded sequential structure via their computation graph—a strategy that leads to well-known pathologies such as the inability to model long-range dependencies [20].

There is a growing trend across deep learning towards more flexible architectures, which avoid strict and structural inductive biases. Most notably, the exceptionally successful Transformer architecture removes any structural inductive bias by encoding the structure via soft inductive biases, such as positional encodings [36]. In the context of GNNs, the self-attention mechanism of a Transformer can be viewed as passing messages between all nodes, regardless of the input graph connectivity.

Prior work has proposed to use attention in GNNs in different ways. First, the GAT model [37] proposed local attention on pairs of nodes that allows a learnable convolutional kernel. The GTN work [42] has improved on the GAT for node and link predictions while keeping a similar architecture, while other message-passing approaches have used enhancing spectral features [8, 13]. More recently, the GT model [14] was proposed as a generalization of Transformers to graphs, where they experimented with sparse and full graph attention while providing low-frequency eigenvectors of the Laplacian as positional encodings.

In this work, we offer a principled investigation of how Transformer architectures can be applied in graph representation learning. **Our primary contribution** is the development of novel and powerful learnable positional encoding methods, which are rooted in spectral graph theory. Our positional encoding technique — and the resulting *spectral attention network (SAN)* architecture — addresses key theoretical limitations in prior graph Transformer work [14] and provably exceeds the expressive power of standard message-passing GNNs. We show that full Transformer-style attention provides consistent empirical gains compared to an equivalent sparse message-passing model, and we demonstrate that our SAN architecture is competitive with or exceeding the state-of-the-art on several well-known graph benchmarks. An overview of the entire method is presented in Figure 1, with a link to the code here: <https://github.com/DevinKreuzer/SAN>.

2 Theoretical Motivations

There can be a significant loss in structural information if naively generalizing Transformers to graphs. To preserve this information as well as local connectivity, previous studies [37, 14] have proposed to use the eigenfunctions of their Laplacian as positional encodings. Taking this idea further by using the full expressivity of eigenfunctions as positional encodings, we can propose a principled way of understanding graph structures using their spectra. The advantages of our methods compared to previous studies [37, 14] are shown in Table 1.

Table 1: Comparison of the properties of different graph Transformer models.

MODELS	GAT [37]	GT sparse [14]	GT full [14]	SAN (Node LPE)
Preserves local structure in attention	✓	✓	✗	✓
Uses edge features	✗	✓	✗	✓
Connects non-neighbouring nodes	✗	✗	✓	✓
Uses eigenvector-based PE for attention	✗	✓	✓	✓
Use a PE with structural information	✗	✓	✗ ²	✓
Considers the ordering of the eigenvalues	✗	✓	✓	✓
Invariant to the norm of the eigenvector	-	✓	✓	✓
Considers the spectrum of eigenvalues	✗	✗	✗	✓
Considers variable # of eigenvectors	-	✗	✗	✓
Aware of eigenvalue multiplicities	-	✗	✗	✓
Invariant to the sign of the eigenvectors	-	✗	✗	✗

¹Presented results add full connectivity before computing the eigenvectors, thus losing the structural information of the graph.

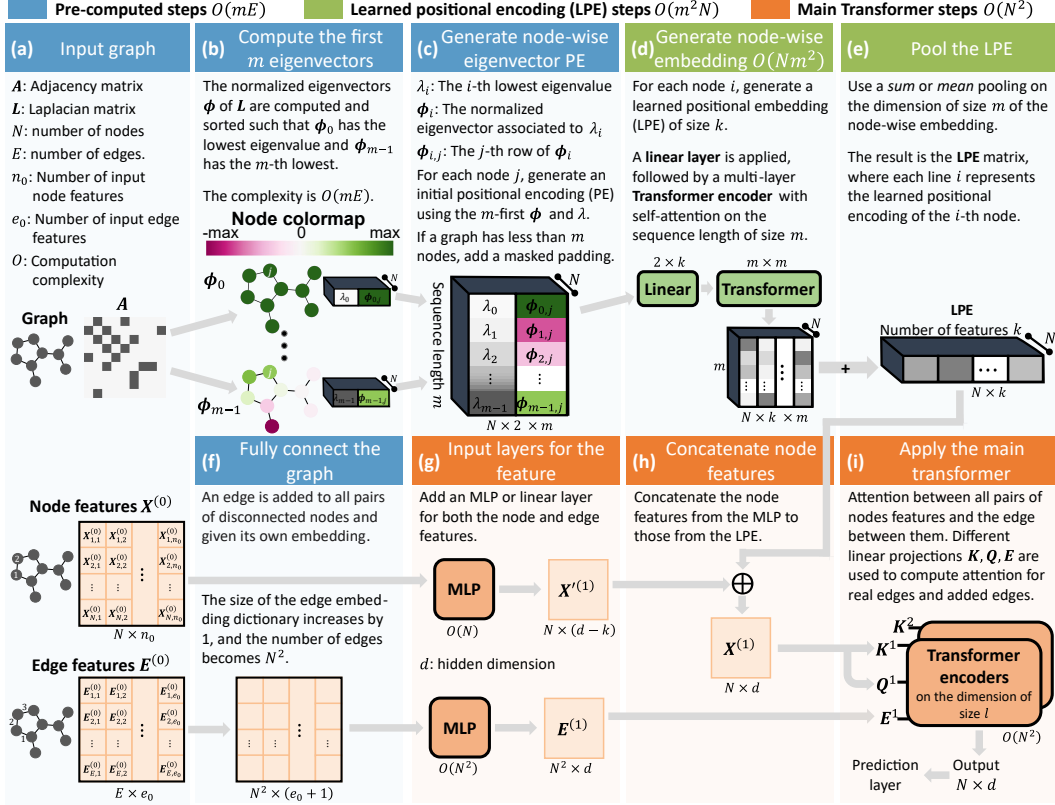


Figure 1: The proposed SAN model with the node LPE, a generalization of Transformers to graphs.

2.1 Absolute and relative positional encoding with eigenfunctions

The notion of positional encodings (PEs) in graphs is not a trivial concept, as there exists no canonical way of ordering nodes or defining axes. In this section, we investigate how eigenfunctions of the Laplacian can be used to define absolute and relative PEs in graphs, to measure physical interactions between nodes, and to enable "hearing" of specific sub-structures - similar to how the sound of a drum can reveal its structure.

2.1.1 Eigenvectors equate to sine functions over graphs

In the Transformer architecture, a fundamental aspect is the use of sine and cosine functions as PEs for sequences [36]. However, sinusoids cannot be clearly defined for arbitrary graphs, since there is no clear notion of position along an axis. Instead, their equivalent is given by the eigenvectors ϕ of the graph Laplacian L . Indeed, in a Euclidean space, the Laplacian (or Laplace) operator corresponds to the divergence of the gradient and its eigenfunctions are sine/cosine functions, with the squared frequencies corresponding to the eigenvalues (we sometimes interchange the two notions from here on). Hence, in the graph domain, the eigenvectors of the graph Laplacian are the natural equivalent of sine functions, and this intuition was employed in multiple recent works which use the eigenvectors as PEs for GNNs [15], for directional flows [4] and for Transformers [14].

Being equivalent to sine functions, we naturally find that the Fourier Transform of a function $\mathcal{F}[f]$ applied to a graph gives $\mathcal{F}[f](\lambda_i) = \langle f, \phi_i \rangle$, where the eigenvalue is considered as a position in the Fourier domain of that graph [6]. Thus, the eigenvectors are best viewed as vectors positioned on the axis of eigenvalues rather than components of a matrix as illustrated in Figure 2.

2.1.2 What do eigenfunctions tell us about relative positions?

In addition to being the analog of sine functions, the eigenvectors of the Laplacian also hold important information about the physics of a system and can reveal distance metrics. This is not surprising as

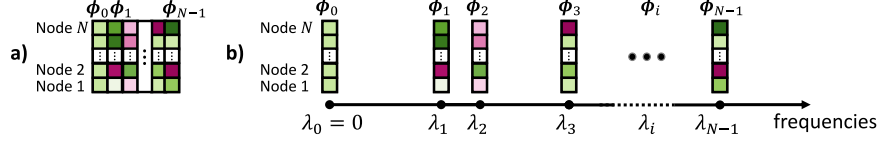


Figure 2: a) Standard view of the eigenvectors as a matrix. b) Eigenvectors ϕ_i viewed as vectors positioned on the axis of frequencies (eigenvalues).

the Laplacian is a fundamental operator in physics and is notably used in Maxwell's equations [16] and the heat diffusion [6].

In electromagnetic theory, the (pseudo)inverse of the Laplacian, known in mathematics as the Green's function of the Laplacian [9], represents the electrostatic potential of a given charge. In a graph, the same concept uses the pseudo-inverse of the Laplacian $\mathbf{G}$ and can be computed by its eigenfunctions. See equation 1, where $\mathbf{G}(j_1, j_2)$ is the electric potential between nodes j_1 and j_2 , ϕ_i and $\hat{\lambda}_i$ are the i -th eigenvectors and eigenvalues of the symmetric Laplacian $\mathbf{D}^{-\frac{1}{2}} \mathbf{L} \mathbf{D}^{-\frac{1}{2}}$, and $\mathbf{D}$ is the degree matrix, and $\hat{\phi}_{i,j}$ the j -th row of the vector.

$$\mathbf{G}(j_1, j_2) = d_{j_1}^{\frac{1}{2}} d_{j_2}^{-\frac{1}{2}} \sum_{i>0} \frac{(\hat{\phi}_{i,j_1} \hat{\phi}_{i,j_2})^2}{\hat{\lambda}_i} \quad (1)$$

Further, the original solution of the heat equation given by Fourier relied on a sum of sines/cosines known as a Fourier series [7]. As eigenvectors of the Laplacian are the analogue of these functions in graphs, we find similar solutions. Knowing that heat kernels are correlated to random walks [6, 4], we use the interaction between two heat kernels to define in equation 2 the diffusion distance d_D between nodes j_1, j_2 [6, 10]. Similarly, the biharmonic distance d_B was proposed as a better measure of distances [28]. Here we use the eigenfunctions of the regular Laplacian $\mathbf{L}$.

$$d_D^2(j_1, j_2) = \sum_{k>0} e^{-2t\lambda_k} (\phi_{k,j_1} - \phi_{k,j_2})^2, \quad d_B^2(j_1, j_2) = \sum_{i>0} \frac{(\phi_{i,j_1} - \phi_{i,j_2})^2}{\lambda_i^2} \quad (2)$$

There are a few things to note from these equations. Firstly, they highlight the importance of pairing *eigenvectors and their corresponding eigenvalues* when supplying information about relative positions in a graph. Secondly, we notice that the product of eigenvectors is proportional to the electrostatic interaction, while the subtraction is proportional to the diffusion and biharmonic distances. Lastly, there is a consistent pattern across all 3 equations: smaller frequencies/eigenvalues are more heavily weighted when determining distances between nodes.

2.1.3 Hearing the shape of a graph and its sub-structures

Another well-known property of eigenvalues is how they can be used to discriminate between different graph structures and sub-structures, as they can be interpreted as the frequencies of resonance of the graph. This led to the famous question about whether we can hear the shape of a drum from its eigenvalues [23], with the same questions also applying to geometric objects [12] and 3D molecules [33]. Various success was found with the eigenfunctions being used for partial functional correspondence [32], algorithmic understanding geometries [26], and style correspondence [12]. Examples of eigenvectors for molecular graphs are presented in Figure 3.

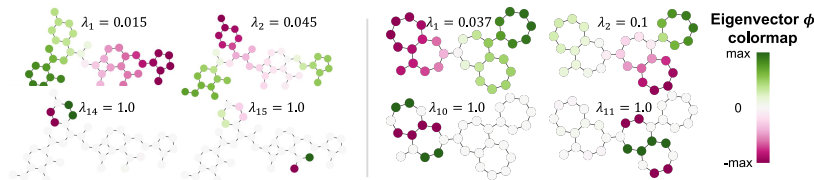


Figure 3: Examples of eigenvalues λ_i and eigenvectors ϕ_i for molecular graphs. The low-frequency eigenvectors ϕ_1, ϕ_2 are spread across the graph, while higher frequencies, such as ϕ_{14}, ϕ_{15} for the left molecule or ϕ_{10}, ϕ_{11} for the right molecule, often resonate in local structures.

2.2 Laplace Eigenfunctions *etiquette*

In Euclidean space and sequences, using sinusoids as PEs is trivial: we can simply select a set of frequencies, compute the sinusoids, and add or concatenate them to the input embeddings, as is done in the original Transformer [36]. However, in arbitrary graphs, reproducing these steps is not as simple since each graph has a unique set of eigenfunctions. In the following section, we present key principles from spectral graph theory to consider when constructing PEs for graphs, most of which have been overlooked by prior methods. They include normalization, the importance of the eigenvalues and their multiplicities, the number of eigenvectors being variable, and sign ambiguities. Our LPE architectures, presented in section 3, aim to address them.

Normalization. Given an eigenvalue of the Laplacian, there is an associated eigenspace of dimension greater than 1. To make use of this information in our model, a single eigenvector has to be chosen. In our work, we use the L_2 normalization since it is compatible with the definition of the Green’s function (1). Thus, we will always chose eigenvectors ϕ such that $\langle \phi, \phi \rangle = 1$.

Eigenvalues. Another fundamental aspect is that the eigenvalue associated with each eigenvector supplies valuable information. An ordering of the eigenvectors based on their eigenvalue works in sequences since the frequencies are pre-determined. However, this assumption does not work in graphs since the eigenvalues in their spectrum can vary. For example, in Figure 3, we observe how an ordering would miss the fact that both molecules resonate at $\lambda = 1$ in different ways.

Multiplicities. Another important problem with choosing eigenfunctions is the possibility of a high multiplicity of the eigenvalues, i.e. when an eigenvalue appears as a root of the characteristic polynomial more than once. In this case, the associated eigenspace may have dimension 2 or more as we can generate a valid eigenvector from any linear combination of eigenvectors with the same eigenvalue. This further complicates the problem of choosing eigenvectors for algorithmic computations and highlights the importance of having a model that can handle this ambiguity.

Variable number of eigenvectors. A graph G_i can have at most N_i linearly independent eigenvectors with N_i being its number of nodes. Most importantly, N_i can vary across all G_i in the dataset. Prior work [14] elected to select a fixed number k eigenvectors for each graph, where $k \leq N_i, \forall i$. This produces a major bottleneck when the smallest graphs have significantly fewer nodes than the largest graphs in the dataset since a very small proportion of eigenvectors will be used for large graphs. This inevitably causes loss of information and motivates the need for a model which constructs fixed PEs of dimension k , where k does not depend on the number of eigenvectors in the graph.

Sign invariance. As noted earlier, there is a sign ambiguity with the eigenvectors. With the sign of ϕ being independent of its normalization, we are left with a total of 2^k possible combination of signs when choosing k eigenvectors of a graph. Previous work has proposed to do data augmentation by randomly flipping the sign of the eigenvectors [4, 15, 14], and although it can work when k is small, it becomes intractable for large k .

2.3 Learning with Eigenfunctions

Learning generalizable information from eigenfunctions is fundamental to their succesful usage. Here we detail important points that support it is possible to do so if done correctly.

Similar graphs have similar spectra. Thus, we can expect the network to transfer patterns across graphs through the similarity of their spectra. In fact, spectral graph theory tells us that the lowest and largest non-zero eigenvalues are both linked to the geometry of the graph (algebraic connectivity and spectral radius).

Eigenspaces contain geometric information. Spectral graph theory has studied the geometric and physical properties of graphs from their Laplacian eigenfunctions in depth. Developing a method that can use the full spectrum of a graph makes it theoretically possible capture this information. It us thus important to capture differences between the full eigenspaces instead of minor differences between specific eigenvalues or eigenvectors from graph to graph.

Learned positions are relative within graphs. Eigenspaces are used to understand the relationship between nodes within graphs, not across them. Proposed models should therefore only compare the eigenfunctions of nodes within graphs.

3 Model Architecture

In this section, we propose an elegant architecture that can use the eigenfunctions as PEs while addressing the concerns raised in section 2.2. Our *Spectral Attention Network* (SAN) model inputs eigenfunctions of a graph and projects them into a learned positional encoding (LPE) of fixed size. The LPE allows the network to use up to the entire Laplace spectrum of each graph, learn how the frequencies interact, and decide which are most important for the given task.

We propose a two-step learning process summarized earlier in Figure 1. The first step, depicted by blocks (c-d-e) in the figure, applies a Transformer over the eigenfunctions of each node to generate an LPE matrix for each graph. The LPE is then concatenated to the node embeddings (blocks g-h), before being passed to the Graph Transformer (block i). If the task involves graph classification or regression, the final node embeddings are subsequently passed to a final pooling layer.

3.1 LPE Transformer Over Nodes

Using Laplace encodings as node features is ubiquitous in the literature concerning the topic. Here, we propose a method for learning node PEs motivated by the principles from section 2.2. The idea of our LPE is inspired by Figure 2, where the eigenvectors ϕ are represented as a non-uniform sequence with the eigenvalue λ being the position on the frequency axis. With this representation, Transformers are a natural choice for processing them and generating a fixed-size PE.

The proposed LPE architecture is presented in Figure 4. First, we create an embedding matrix of size $2 \times m$ for each node j by concatenating the m -lowest eigenvalues with their associated eigenvectors. Here, m is a hyper-parameter for the maximum number of eigenvectors to compute and is analog to the variable-length sequence for a standard Transformer. For graphs where $m > N$, a masked-padding is simply added. Note that to capture the entire spectrum of all graphs, one can simply select m such that it is equal to the maximum number of nodes a graph has in the dataset. A linear layer is then applied on the dimension of size 2 to generate new embeddings of size k . A Transformer Encoder then computes self-attention on the sequence of length m and hidden dimension k . Finally, a sum pooling reduces the sequence into a fixed k -dimensional node embedding.

The LPE model addresses key limitations of previous graph Transformers and is aligned with the first four *etiquettes* presented in section 2.2. By concatenating the eigenvalues with the normalized eigenvector, this model directly addresses the first three *etiquettes*. Namely, it **normalizes** the eigenvectors, pairs eigenvectors with their **eigenvalues** and treats **the number of eigenvectors as a variable**. Furthermore, the model is aware of **multiplicities** and has the potential to linearly combine or ignore some of the repeated eigenvalues.

However, this method still does not address the limitation that the sign of the pre-computed eigenvectors is arbitrary. To combat this issue, we randomly flip the sign of the pre-computed eigenvectors during training as employed by previous work [15, 14], to promote invariance to the sign ambiguity.

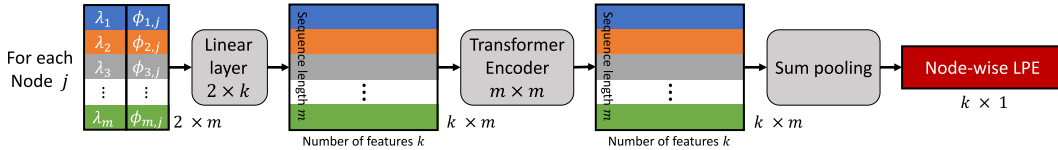


Figure 4: Learned positional encoding (LPE) architectures, with the model being aware of the graph’s Laplace spectrum by considering m eigenvalues and eigenvectors, where we permit $m \leq N$, with N denoting the number of nodes. Since the Transformer loops over the nodes, each node can be viewed as an element of a batch to parallelize the computation. Here $\phi_{i,j}$ is the j -th element of the eigenvector paired to the i -th lowest eigenvalue λ_i .

3.2 LPE Transformer Over Edges

Here we present an alternative formulation for Laplace encodings. This method addresses the same issues as the LPE over nodes, but also resolves the eigenvector sign ambiguity. Instead of encoding *absolute* positions as node features, the idea is to consider *relative* positions encoded as edge features.

Inspired by the physical interactions introduced in 1 and 2, we can take a pair of nodes (j_1, j_2) and obtain **sign-invariant** operators using the absolute subtraction $|\phi_{i,j_1} - \phi_{i,j_2}|$ and the product $\phi_{i,j_1} \phi_{i,j_2}$. These operators acknowledge that the sign of ϕ_{i,j_1} at a given node j_1 is not important, but that the relative sign between nodes j_1 and j_2 is important. One might argue that we could directly compute the deterministic values from equations (1, 2) as edge features instead. However, our goal is to construct models that can learn which frequencies to emphasize and are not biased towards the lower frequencies — despite lower frequencies being useful in many tasks.

This approach is only presented thoroughly in appendix A, since it suffers from a major computational bottleneck compared to the LPE over nodes. In fact, for a fully-connected graph, there are N times more edges than nodes, thus the computation complexity is $O(m^2 N^2)$, or $O(N^4)$ considering all eigenfunctions. The same limitation also affects memory and prevents the use of large batch sizes.

3.3 Main Graph Transformer

Our attention mechanism in the main Transformer is based on previous work [14], which attempts to repurpose the original Transformer to graphs by considering the graph structure and improving attention estimates with edge feature embeddings.

In the following, note that $\mathbf{h}_i^l$ is the i -th node’s features at the l -th layer, and $\mathbf{e}_{ij}$ is the edge feature embedding between nodes i and j . Our model employs multi-head attention over all nodes:

$$\hat{\mathbf{h}}_i^{l+1} = \mathbf{O}_h^l \parallel \left(\sum_{k=1}^H \sum_{j \in V} w_{ij}^{k,l} \mathbf{V}^{k,l} \mathbf{h}_j^l \right) \quad (3)$$

where $\mathbf{O}_h^l \in \mathbb{R}^{d \times d}$, $\mathbf{V}^{k,l} \in \mathbb{R}^{d_k \times d}$, H denotes the number of heads, L the number of layers, and $\parallel$ concatenation. Note that d is the hidden dimension, while d_k is the dimension of a head ($\frac{d}{H} = d_k$).

A key addition from our work is the design of an architecture that performs full-graph attention while preserving local connectivity with edge features via two sets of attention mechanisms: one for nodes connected by real edges in the sparse graph and one for nodes connected by added edges in the fully-connected graph. The attention weights $w_{ij}^{k,l}$ in equation 3 at layer l and head k are given by:

$$\hat{w}_{ij}^{k,l} = \begin{cases} \frac{\mathbf{Q}^{1,k,l} \mathbf{h}_i^l \circ \mathbf{K}^{1,k,l} \mathbf{h}_j^l \circ \mathbf{E}^{1,k,l} \mathbf{e}_{ij}}{\sqrt{d_k}} & \text{if } i \text{ and } j \text{ are connected in sparse graph} \\ \frac{\mathbf{Q}^{2,k,l} \mathbf{h}_i^l \circ \mathbf{K}^{2,k,l} \mathbf{h}_j^l \circ \mathbf{E}^{2,k,l} \mathbf{e}_{ij}}{\sqrt{d_k}} & \text{otherwise} \end{cases} \quad (4)$$

$$w_{ij}^{k,l} = \begin{cases} \frac{1}{1+\gamma} \cdot \text{softmax}(\sum_{d_k} \hat{w}_{ij}^{k,l}) & \text{if } i \text{ and } j \text{ are connected in sparse graph} \\ \frac{\gamma}{1+\gamma} \cdot \text{softmax}(\sum_{d_k} \hat{w}_{ij}^{k,l}) & \text{otherwise} \end{cases} \quad (5)$$

where $\circ$ denotes element-wise multiplication and $\mathbf{Q}^{1,k,l}, \mathbf{Q}^{2,k,l}, \mathbf{K}^{1,k,l}, \mathbf{K}^{2,k,l}, \mathbf{E}^{1,k,l}, \mathbf{E}^{2,k,l} \in \mathbb{R}^{d_k \times d}$. $\gamma \in \mathbb{R}^+$ is a hyperparameter which tunes the amount of bias towards full-graph attention, allowing flexibility of the model to different datasets and tasks where the necessity to capture long-range dependencies may vary. Note that softmax outputs are clamped between -5 and 5 for numerical stability and that the keys, queries and edge projections are different for pairs of connected nodes ($\mathbf{Q}^1, \mathbf{K}^1, \mathbf{E}^1$) and disconnected nodes ($\mathbf{Q}^2, \mathbf{K}^2, \mathbf{E}^2$).

A multi-layer perceptron (MLP) with residual connections and normalization layers are then applied to update representations, in the same fashion as the GT method [14].

$$\hat{\mathbf{h}}^{l+1} = \text{Norm}(\mathbf{h}_i^l + \hat{\mathbf{h}}_i^{l+1}), \quad \hat{\mathbf{h}}_i^{l+1} = \mathbf{W}_2^l \text{ReLU}(\mathbf{W}_1^l \hat{\mathbf{h}}_i^{l+1}), \quad \mathbf{h}_i^{l+1} = \text{Norm}(\hat{\mathbf{h}}^{l+1} + \hat{\mathbf{h}}_i^{l+1}) \quad (6)$$

with the weight matrices $\mathbf{W}_1^l \in \mathbb{R}^{2d \times d}$, $\mathbf{W}_2^l \in \mathbb{R}^{d \times 2d}$. Edge representations are not updated as it adds complexity with little to no performance gain. Bias terms are omitted for presentation.

3.4 Limitations

The first limitation of the node-wise LPE, and noted in Table 1 is the lack of sign invariance of the model. A random sign-flip of an eigenvector can produce different outputs for the LPE, meaning

that the model needs to learn a representation invariant to these flips. We resolve this issue with the edge-wise LPE proposed in 3.2, but it comes at a computational cost.

Another limitation of the approach is the computational complexity of the LPE being $O(m^2N)$, or $O(N^3)$ if considering all eigenfunctions. Further, as nodes are batched in the LPE, the total memory on the GPU will be $\text{num_params} * \text{num_nodes_in_batch}$ instead of $\text{num_params} * \text{batch_size}$. Although this is limiting, the LPE is not parameter hungry, with k usually kept around 16. Most of the model’s parameters are in the *Main Graph Transformer* of complexity $O(N^2)$.

Despite Transformers having increased complexity, they managed to revolutionize the NLP community. We argue that to shift away from the message-passing paradigm and generalize Transformers to graphs, it is natural to expect higher computational complexities. This is exacerbated by sequences being much simpler to understand than graphs due to their linear structure. Future work could overcome this by using variations of Transformers that scale linearly or logarithmically [34].

3.5 Theoretical properties of the architecture

Due to the full connectivity, it is trivial that our model does not suffer from the same limitations in expressivity as its convolutional/message-passing counterpart.

WL test and universality. The DGN paper [4] showed that using the eigenvector ϕ_1 is enough to distinguish some non-isomorphic graphs indistinguishable by the 1-WL test.

Given that our model uses the full set of eigenfunctions, and given enough parameters, our model can distinguish any pair of non-isomorphic graphs and is more powerful than any WL test in that regard. However, this does not solve the graph isomorphism problem in polynomial time; it only approximates a solution, and the number of parameters required is unknown and possibly non-polynomial. In appendix C, we present a proof of our statement, and discuss why the WL test is not well suited to study the expressivity of graph Transformers due to their universality.

Reduced over-squashing. Over-squashing represents the difficulty of a graph neural network to pass information to distant neighbours due to the exponential blow-up in computational paths [1].

For the fully-connected network, it is trivial to see that over-squashing is non-existent since there are direct paths between distant nodes.

Physical interactions. Another point to consider is the ability of the network to learn physical interactions between nodes. This is especially important when the graph models physical, chemical, or biological structures, but can also help understanding pixel interaction in images [2, 3]. Here, we argue that our SAN model, which uses the Laplace spectrum more effectively, can learn to mimic the physical interactions presented in section 2.1.2. This contrasts with the convolutional approach that requires deep layers for the receptive field to capture long-distance interactions. It also contrasts with the GT model [14], which does not use eigenvalues or enough eigenfunctions to properly model physical interactions in early layers. However, due to the lack of sign-invariance in the proposed node-wise LPE, it is difficult to learn these interactions accurately. The edge-wise LPE (section 3.2) could be better suited for the problem, but it suffers from higher computational complexity.

4 Experimental Results

The model is implemented in PyTorch [31] and DGL [38] and tested on established benchmarks from [15] and [21] provided under MIT license. Specifically, we applied our method on ZINC, PATTERN, CLUSTER, MolHIV and MolPCBA, while following their respective training protocols with minor changes, as detailed in the appendix B.1. The computation time and hardware is provided in appendix B.4.

We first conducted an ablation study to fairly compare the benefits of using full attention and/or the node LPE. We then took the best-performing model, tuned some of its hyperparameters, and matched it up against the current state-of-the-art methods. Since we use a similar attention mechanism, our code was developed on top of the code from the GT paper [14], provided under the MIT license.

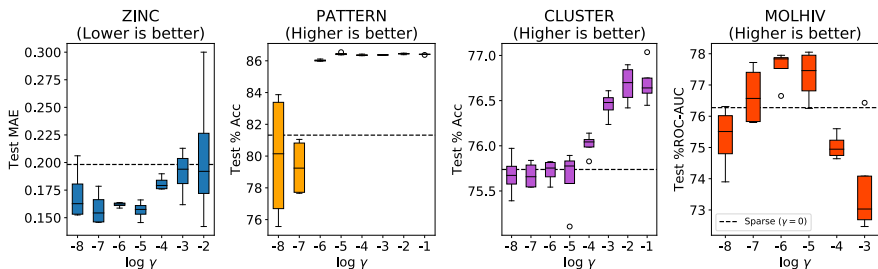


Figure 5: Effect of the γ parameter on the performance across datasets from [15, 21], using the Node LPE. Dotted black lines indicate sparse attention, which is equivalent to setting $\gamma = 0$. Each box plot consists of 4 runs, with different seeds (except MolHIV).

Model details		ZINC	PATTERN	CLUSTER	MOLHIV	Best Worst
Attention	LPE	MAE	% ACC	% ACC	% ROC-AUC	
Sparse	-	0.267 $\pm$ 0.032	83.613 $\pm$ 0.663	75.683 $\pm$ 0.098	73.46 $\pm$ 0.71	
Sparse	Node	0.198 $\pm$ 0.004	81.329 $\pm$ 2.150	75.738 $\pm$ 0.106	76.61 $\pm$ 0.62	
Full	-	0.392 $\pm$ 0.055	86.322 $\pm$ 0.049	76.447 $\pm$ 0.177	73.84 $\pm$ 1.80	
Full	Node	0.157 $\pm$ 0.006	86.441 $\pm$ 0.040	76.691 $\pm$ 0.247	77.57 $\pm$ 0.61	

Figure 6: Ablation study on datasets from [15, 21] for the node LPE and full graph attention, with no hyperparameter tuning other than γ taken from Figure 5. For a given dataset, all models use the same hyperparameters, but the hidden dimensions are adjusted to have $\sim 500k$ learnable parameters. Means and uncertainties are derived from four runs, with different seeds (except MolHIV).

4.1 Sparse vs. Full Attention

To study the effect of incorporating full attention, we present an ablation study of the γ parameter in Figure 5. We remind readers that γ is used in equation 5 to balance between *sparse* and *full* attention. Setting $\gamma = 0$ strictly enables sparse attention, while $\gamma = 1$ does not bias the model in any direction.

It is apparent that molecular datasets, namely ZINC and MOLHIV, benefit less from full attention, with the best parameter being $\log \gamma \in (-7, -5)$. On the other hand, the larger SBM datasets (PATTERN and CLUSTER) benefit from a higher γ value. This can be explained by the fact that molecular graphs rely more on understanding local structures such as the presence of rings and specific bonds, especially in the artificial task from ZINC which relies on counting these specific patterns [15]. Furthermore, molecules are generally smaller than SBMs. As a result, we would expect less need for full attention, as information between distant nodes can be propagated with few iterations of even sparse attention. We also expect molecules to have fewer multiplicities, thus reducing the space of eigenvectors. Lastly, the performance gains in using full attention on the CLUSTER dataset can be attributed to it being a semi-supervised task, where some nodes within each graph are assigned their true labels. With full attention, every node receives information from the labeled nodes at each iteration, reinforcing confidence about the community they belong to.

In Figure 6, we present another ablation study to measure the impact of the node LPE in both the *sparse* and *full* architectures. We observe that the proposed node-wise LPE contributes significantly to the performance for molecular tasks (ZINC and MOLHIV), and believe that it can be attributed to the detection of substructures (see Figure 3). For PATTERN and CLUSTER, the improvement is modest as the tasks are simple clustering [15]. Previous work even found that the optimal number of eigenvectors to construct PE for PATTERN is only 2 [14].

4.2 Comparison to the state-of-the-art

When comparing to the state-of-the-art (SOTA) models in the literature in Figure 7, we observe that our SAN model consistently performs better on all synthetic datasets from [15], highlighting the strong expressive power of the model. On the MolHIV dataset, the performance on the test set is slightly lower than the SOTA. However, the model performs better on the validation set (85.30%) in comparison to PNA (84.25%) and DGN (84.70%). This can be attributed to a well-known issue with this dataset: the validation and test metrics have low correlation. In our experiments, we found higher test results with lower validation scores when restricting the number of epochs. Here, we also included results on the MolPCBA dataset, where we witnessed competitive results as well.

Other top-performing models, namely PNA [11] and DGN [4], use a message-passing approach [17] with multiple aggregators. When compared to attention-based models, SAN consistently outperforms the SOTA by a wide margin. To the best of our knowledge, SAN is the first fully-connected model to perform well on graph tasks, as is evident by the poor performance of the *GT (full)* model.

	ZINC	PATTERN	CLUSTER	MOLHIV	MOLPCBA
Model	MAE	% Acc	% Acc	% ROC-AUC	% AP
GCN	0.367 $\pm$ 0.011	71.892 $\pm$ 0.334	68.498 $\pm$ 0.976	76.06 $\pm$ 0.97	20.20 $\pm$ 0.24
GraphSage	0.398 $\pm$ 0.002	50.492 $\pm$ 0.001	63.844 $\pm$ 0.110	-	-
GatedGCN	0.282 $\pm$ 0.015	85.568 $\pm$ 0.088	73.840 $\pm$ 0.326	-	-
GatedGCN-PE	0.214 $\pm$ 0.013	86.508 $\pm$ 0.085	76.082 $\pm$ 0.196		
GIN	0.526 $\pm$ 0.013	85.387 $\pm$ 0.136	64.716 $\pm$ 1.553	75.58 $\pm$ 1.40	22.66 $\pm$ 0.28
PNA	0.142 $\pm$ 0.010	-	-	79.05 $\pm$ 1.32	28.38 $\pm$ 0.35
DGN	-	-	-	79.70 $\pm$ 0.97	28.85 $\pm$ 0.30
Attention-based					
GAT	0.384 $\pm$ 0.007	78.271 $\pm$ 0.186	70.587 $\pm$ 0.447	-	-
GT (sparse)	0.226 $\pm$ 0.014	84.808 $\pm$ 0.068	73.169 $\pm$ 0.662	-	-
GT (full)	0.598 $\pm$ 0.049	56.482 $\pm$ 3.549	27.121 $\pm$ 8.471	-	-
SAN	0.139 $\pm$ 0.006	86.581 $\pm$ 0.037	76.691 $\pm$ 0.65	77.85 $\pm$ 0.247	27.65 $\pm$ 0.42

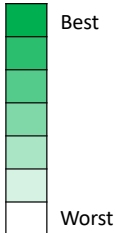


Figure 7: Comparing our tuned model on datasets from [15, 21], against GCN [25], GraphSage [18], GIN [39], GAT [37], GatedGCN [5], PNA [11], and DGN [4]. Means and uncertainties are derived from four runs with different seeds, except MolHIV which uses 10 runs with identical seed. The number of parameters is fixed to $\sim 500k$ for ZINC, PATTERN and CLUSTER.

5 Conclusion

In summary, we presented the SAN model for graph neural networks, a new Transformer-based architecture that is aware of the Laplace spectrum of a given graph from the learned positional encodings. The model was shown to perform on par or better than the SOTA on multiple benchmarks and outperforms other Attention-based models by a large margin. As is often the case with Transformers, the current model suffers from a computational bottleneck, and we leave it for future work to implement variations of Transformers that scale linearly or logarithmically. This will enable the edge-wise LPE presented in appendix A, a theoretically more powerful version of the SAN model.

Societal Impact. The presented work is focused on theoretical and methodological improvements to graph neural networks, so there are limited direct societal impacts. However, indirect negative impacts could be caused by malicious applications developed using the algorithm. One such example is the tracking of people on social media by representing their interaction as graphs, thus predicting and influencing their behavior towards an external goal. It also has an environmental impact due to the greater energy use that arises from the computational cost $O(m^2N + N^2)$ being larger than standard message passing or convolutional approaches of $O(E)$.

Funding Disclosure.

Devin Kreuzer is supported by an NSERC grant.

References

- [1] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. *arXiv:2006.05205 [cs, stat]*, 2020.
- [2] Dominique Beaini, Sofiane Achiche, Alexandre Duperré, and Maxime Raison. Deep green function convolution for improving saliency in convolutional neural networks. *The Visual Computer*, 37(2):227–244, 2020.
- [3] Dominique Beaini, Sofiane Achiche, and Maxime Raison. Improving convolutional neural networks via conservative field regularisation and integration.

- [4] Dominique Beaini, Saro Passaro, Vincent Létourneau, William L. Hamilton, Gabriele Corso, and Pietro Liò. Directional graph networks. *ICML2021*, 2021.
- [5] Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.
- [6] Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4):18–42, 2017.
- [7] F. Cajori. *A History of Mathematics*. AMS Chelsea Publishing Series. AMS Chelsea, 1999.
- [8] Heng Chang, Yu Rong, Tingyang Xu, Wenbing Huang, Somayeh Sojoudi, Junzhou Huang, and Wenwu Zhu. Spectral graph attention network. *CoRR*, abs/2003.07450, 2020.
- [9] Fan Chung and S. T. Yau. Discrete green’s functions. *Journal of Combinatorial Theory, Series A*, 91(1):191–214, 2000.
- [10] Ronald R. Coifman and Stéphane Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006. Special Issue: Diffusion Maps and Wavelets.
- [11] Gabriele Corso, Luca Cavalleri, Dominique Beaini, Pietro Liò, and Petar Veličković. Principal neighbourhood aggregation for graph nets. *arXiv preprint arXiv:2004.05718*, 2020.
- [12] Luca Cosmo, Mikhail Panine, Arianna Rampini, Maks Ovsjanikov, Michael M. Bronstein, and Emanuele Rodola. Isospectralization, or how to hear shape, style, and correspondence. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- [13] Yushun Dong, Kaize Ding, Brian Jalaian, Shuiwang Ji, and Jundong Li. Graph neural networks with adaptive frequency response filter. *CoRR*, abs/2104.12840, 2021.
- [14] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs, 2020.
- [15] Vijay Prakash Dwivedi, Chaitanya K Joshi, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *arXiv preprint arXiv:2003.00982*, 2020.
- [16] Richard Phillips Feynman, Robert Benjamin Leighton, and Matthew Sands. *The Feynman lectures on physics; New millennium ed.* Basic Books, New York, NY, 2010. Originally published 1963-1965.
- [17] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 1263–1272. JMLR. org, 2017.
- [18] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.
- [19] William L. Hamilton. *Graph Representation Learning*. Morgan and Claypool, 2020.
- [20] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [21] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *arXiv preprint arXiv:2005.00687*, 2020.
- [22] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. *arXiv:1802.04364 [cs, stat]*, 2018.
- [23] Mark Kac. Can one hear the shape of a drum? *The American Mathematical Monthly*, 73(4):1, 1966.

- [24] Anees Kazi, Luca Cosmo, Nassir Navab, and Michael Bronstein. Differentiable graph module (dgm) graph convolutional networks. *arXiv preprint arXiv:2002.04999*, 2020.
- [25] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [26] B. Levy. Laplace-beltrami eigenfunctions towards an algorithm that "understands" geometry. In *IEEE International Conference on Shape Modeling and Applications 2006 (SMI'06)*, pages 13–13, 2006.
- [27] Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. Distance encoding: Design provably more powerful neural networks for graph representation learning, 2020.
- [28] Yaron Lipman, Raif M. Rustamov, and Thomas A. Funkhouser. Biharmonic distance. *ACM Trans. Graph.*, 29(3), July 2010.
- [29] Haggai Maron, Heli Ben-Hamu, Hadar Serviansky, and Yaron Lipman. Provably powerful graph networks. *arXiv preprint arXiv:1905.11136*, 2019.
- [30] Christopher Morris, Martin Ritzert, Matthias Fey, William L Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019.
- [31] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- [32] Emanuele Rodolà, Luca Cosmo, Michael M. Bronstein, Andrea Torsello, and Daniel Cremers. Partial functional correspondence, 2015.
- [33] Joshua Schrier. Can one hear the shape of a molecule (from its coulomb matrix eigenvalues)? *Journal of Chemical Information and Modeling*, 60(8):3804–3811, 2020. Publisher: American Chemical Society.
- [34] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey, 2020.
- [35] Edwin R. van Dam and Willem H. Haemers. Which graphs are determined by their spectrum? *Linear Algebra and its Applications*, 373:241–272, 2003.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [38] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.
- [39] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.
- [40] Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions?, 2020.
- [41] Chulhee Yun, Yin-Wen Chang, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. $\Omega(n)$ connections are expressive enough: Universal approximability of sparse transformers. *CoRR*, abs/2006.04862, 2020.
- [42] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J. Kim. Graph transformer networks, 2020.

A LPE Transformer Over Edges

Consider one of the most fundamental notions in physics; *Potential energy*. Interestingly, potential energy is always measured as a potential difference; it is not an inherent individual property, such as mass. Strikingly, it is also the *relative* Laplace embeddings of two nodes that paint the picture, as a node’s Laplace embedding on its own reveals no information at all. With this in mind, we argue that Laplace positional encodings are more naturally represented as edge features, which encode a notion of *relative* position of the two endpoints in the graph. This can be viewed as a distance encoding, which was shown to improve the performance of node and link prediction in GNNs [27].

The formulation is very similar to the method for learning positional node embeddings. Here, a Transformer Encoder is applied on each graph by treating edges as a batch of variable size and eigenvectors as a variable sequence length. We again compute up to the m -lowest eigenvectors with their eigenvalues but, instead of directly using the eigenvector elements, we compute the following vectors:

$$|\phi_{:,j_1} - \phi_{:,j_2}| \quad (7) \quad \phi_{:,j_1} \circ \phi_{:,j_2} \quad (8)$$

where “:” denotes along all up to m eigenvectors, and $\circ$ denotes element-wise multiplication. Note that these new vectors are completely invariant to sign permutations of the precomputed eigenvectors.

As per the LPE over nodes, the 3-length vectors are expanded with a linear layer to generate embeddings of size k before being input to the Transformer Encoder. The final embeddings are then passed to a sum pooling layer to generate fixed-size edge positional encodings, which are then used to compute attention weights in equation 4.

This method addresses **all etiquettes** raised in section 2.2. However, it suffers from a major computational bottleneck compared to the LPE over nodes. Indeed, for a fully-connected graph, there are N times more edges than nodes, thus the computation complexity is $O(m^2 N^2)$, or $O(N^4)$ considering all eigenfunctions. This same limitation also affects the memory, as efficiently batching the N^2 edges will increase the memory consumption of the LPE by a drastic amount, preventing the model from using large batch sizes and making it difficult to train.

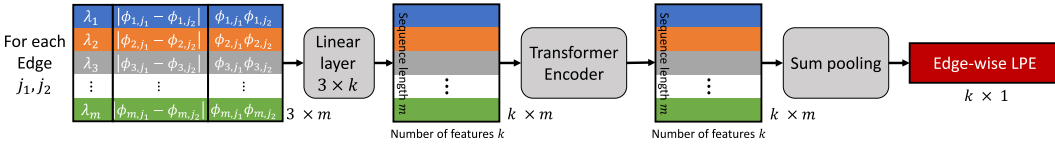


Figure 8: Edge-wise Learned positional encoding (LPE) architectures, where the relative position is considered instead of the absolute position. The model is aware of the graph’s Laplace spectrum by considering m eigenvalues and eigenvectors, where we permit $m \leq N$, with N denoting the number of nodes. Since the Transformer loops over the edges, each edge can be viewed as an element of a batch to parallelize the computation. The computational complexity is $O(m^2 E)$ or $O(m^2 N^2)$ for a fully-connected graph.

B Appendix - Implementation details

B.1 Benchmarks and datasets

To test our models’ performance, we rely on standard benchmarks proposed by [15] and [21] and provided under the MIT license. In particular, we chose ZINC, PATTERN, CLUSTER, and MolHIV.

ZINC [15]. A synthetic molecular graph regression dataset, where the predicted score is given by the subtraction of computationally estimated properties $\log P - SA$. Here, $\log P$ is the computed octanol-water partition coefficient, and SA is the synthetic accessibility score [22].

CLUSTER [15]. A synthetic benchmark for node classification. The graphs are generated with Stochastic Block Models, a type of graph used to model communities in social networks. In total, 6 communities are generated and each community has a single node with its true label assigned. The task is to classify which nodes belong to the same community.

PATTERN [15]. A synthetic benchmark for node classification. The graphs are generated with Stochastic Block Models, a type of graph used to model communities in social networks. The task is to classify the nodes into 2 communities, testing the GNNs ability to recognize predetermined subgraphs.

MolHIV [21]. A real-world molecular graph classification benchmark. The task is to predict whether a molecule inhibits HIV replication or not. The molecules in the training, validation, and test sets are divided using a scaffold splitting procedure that splits the molecules based on their two-dimensional structural frameworks. The dataset is heavily imbalanced towards negative samples. It is also known that this dataset suffers from a strong de-correlation between validation and test set performance, meaning that more hyperparameter fine-tuning on the validation set often leads to lower test set results.

MolPCBA [21]. Another real-world molecular graph classification benchmark. The dataset is larger than MolHIV and applies a similar scaffold splitting procedure. It consists of multiple, extremely skewed (only 1.4% positivity) molecular classification tasks, and employs Average Precision (AP) over them as a metric.

B.2 Ablation studies

The results in Figures 5-6 are done as an ablation study with a minimal tuning of the hyperparameters of the network to measure the impact of the node LPE and full attention. A majority of the hyperparameters used were tuned in previous work [15]. However, we altered some of the existing parameters to accommodate the parameter-heavy LPE, and modified the Main Graph Transformer hidden dimension such that all models have approximately $\sim 500k$ parameters for a fair comparison. We present results for the full attention with the optimal γ value optimized on the Node LPE model. We did this to isolate the impact that the Node LPE has on improving full attention. Details concerning the model architecture parameters are visible in Figure 9.

Attention	LPE	LPE layers	LPE dimension	GT layers	GT hidden dimension	#Parameters
ZINC						
[†] Sparse	-	-	-	6	96	511201
Sparse	Node	3	16	6	72	494865
Full	-	-	-	6	80	471361
Full	Node	3	16	6	64	508577
PATTERN						
Sparse	-	-	-	6	96	508634
Sparse	Node	3	16	6	72	493340
Full	-	-	-	6	80	469142
Full	Node	3	16	6	64	507202
CLUSTER						
Sparse	-	-	-	16	56	461348
Sparse	Node	1	16	16	56	530036
Full	-	-	-	16	48	450498
Full	Node	1	16	16	48	519186
MOLHIV						
Sparse	-	-	-	6	96	525985
Sparse	Node	2	16	6	80	503265
Full	-	-	-	6	80	483601
[†] Full	Node	2	16	6	72	528265

Figure 9: Model architecture parameters for the ablation study. We modify the hidden dimensions of the Main Graph Transformer (GT) such that all models have $\sim 500k$ parameters for a fair comparison. [†]The batch size was doubled to ensure convergence of the model. All other parameters outside the GT hidden dimension are consistent within a dataset experiment.

For the training parameters, we employed an Adam optimizer with a learning rate decay strategy initialized in $\{10^{-3}, 10^{-4}\}$ as per [15], with some minor modifications:

ZINC [15]. We selected an *initial learning rate* of 7×10^{-4} and increased the *patience* from 10 to 25 to ensure convergence. **PATTERN [15]**. We selected an *initial learning rate* of 5×10^{-4} . **CLUSTER [15]**. We selected an *initial learning rate* of 5×10^{-4} and reduced the *minimum learning rate* from 10^{-6} to 10^{-5} to speed up training time. **MolHIV [21]**. We elected to use similar training

procedures for consistency. We selected an *initial learning rate* of 10^{-4} , a *reduce factor* of 0.5, a *patience* of 20, a *minimum learning rate* of 10^{-5} , a *weight decay* of 0 and a *dropout* of 0.03.

B.3 SOTA Comparison study

For the results in Figure 7, we tuned some of the hyperparameters, using the following strategies. The optimal parameters are in **bold**.

ZINC. Due to the 500k parameter budget, we tuned the pairing $\{GT \text{ layers}, GT \text{ hidden dimension}\} \in \{\{6, 72\}, \{8, 64\}, \{\mathbf{10}, \mathbf{56}\}\}$ and $readout \in \{\text{"mean"}, \text{"sum"}\}$ **PATTERN**. Due to the 500k parameter budget and long training times, we only tuned the pairing $\{GT \text{ layers}, GT \text{ hidden dimension}\} \in \{\{\mathbf{4}, \mathbf{80}\}, \{6, 64\}\}$ **CLUSTER**. Due to the 500k parameter budget and long training times, we only tuned the pairing $\{GT \text{ layers}, GT \text{ hidden dimension}\} \in \{\{12, 64\}, \{\mathbf{16}, \mathbf{48}\}\}$ **MolHIV**. With no parameter budget, we elected to do a more extensive parameter tuning in a two-step process while measuring validation metrics on 3 runs with identical seeds.

1. We tuned $LPE \text{ dimension} \in \{8, \mathbf{16}\}$, $GT \text{ layers} \in \{4, 6, 8, \mathbf{10}\}$, $GT \text{ hidden dimension} \in \{48, \mathbf{64}, 72, 80, 96\}$
2. With the highest performing validation model from step 1, we then tuned $dropout \in \{0, \mathbf{0.01}, 0.025\}$ and $weight \text{ decay} \in \{\mathbf{0}, 10^{-6}, 10^{-5}\}$

With the final optimized parameters, we reran 10 experiments with identical seeds.

MolPCBA. With no parameter budget, we elected to do a more extensive parameter tuning as well. We tuned $learning \text{ rate} \in \{0.0001, \mathbf{0.0003}, 0.0005\}$, $dropout \in \{0, 0.1, 0.2, 0.3, 0.4, \mathbf{0.5}\}$, $GT \text{ layers} \in \{2, 4, \mathbf{5}, 6, 8, 10, 12\}$, $GT \text{ layers} \in \{128, 256, \mathbf{304}, 512\}$, $LPE \text{ layers} \in \{8, \mathbf{10}, 12\}$ and $LPE \text{ dimension} \in \{8, \mathbf{16}\}$

B.4 Computation details

Dataset	Resource	Cluster	GPU	Epoch/Total time
ZINC	Compute Canada	Graham	Tesla P100-PCIE (12 GB)	106s/17.88hrs
PATTERN	Compute Canada	Graham	Tesla P100-PCIE (12 GB)	340s/12.52hrs
CLUSTER	Compute Canada	Beluga	Tesla V100-SXM2 (16 GB)	433s/11.30hrs
MOLHIV	Compute Canada	Cedar	Tesla V100-SXM2 (32 GB)	204s/5.34hrs
MOLPCBA	Compute Canada	Cedar	Tesla V100-SXM2 (32 GB)	883s/48.02hrs

Figure 10: Computational details for SOTA Comparison study.

C Expressivity and complexity analysis of graph Transformers

In this section, we discuss how the universality of Transformers translates to graphs when using different node identifiers. Theoretically, this means that by simply labeling each node, Transformers can learn to distinguish any graph, and the WL test is no longer suited to study their expressivity.

Thus, we introduce the notion of learning complexity to better compare each architecture’s ability to understand the space of isomorphic graphs. We apply the complexity analysis to the LPE and show that it can more easily capture the structure of graphs than a naive Transformer.

C.1 Universality of Transformers for sequence-to-sequence approximations

In recent work [40] [41], it was proven that Transformers are universal sequence-to-sequence approximators, meaning that they can encode any function that approximately maps any first sequence into a second sequence when given enough parameters. More formally, they proved the following theorems for the universality of Transformers:

Theorem 1. *For any $1 \leq p < \infty$, $\varepsilon > 0$ and any function $f : \mathbf{R}^{d \times n} \rightarrow \mathbf{R}^{d \times n}$ that is equivariant to permutations of the columns, there is a Transformer g such that the L^p distance between f and g is smaller than ε .*

Let $\mathbf{B}^n$ be the n -dimensional closed ball and denote by $C^0(\mathbf{B}^{d \times n}, \mathbf{R}^{d \times n})$ the set of all continuous functions of the ball to $\mathbf{R}^{d \times n}$. A Transformer with positional encoding g_p is a Transformer g such that to each input $\mathbf{X}$, a fixed learned positional encoding $\mathbf{E}$ is added such that $g_p(\mathbf{X}) = g(\mathbf{X} + \mathbf{E})$.

Theorem 2. *For any $1 \leq p < \infty$, $\varepsilon > 0$ and any function $f \in C^0(\mathbf{B}^{d \times n}, \mathbf{R}^{d \times n})$, there is a Transformer with positional encoding g such that the L^p distance between f and g is smaller than ε .*

C.2 Graph Transformers approximate solutions to the graph isomorphism problem

We now explore the consequences of the previous 2 theorems on the use of Transformers for graph representation learning. We first describe 2 types of Transformers on graphs; one for node and one for edge inputs. They will be used to deduce corollaries of theorems 1 and 2 for graph learning and later comparison with our proposed architecture. Assume now that all nodes of the graphs we consider are given an integer label in $\{1, \dots, N\}$.

The naive edge transformer takes as input a graph represented as a sequence of ordered pairs $((i, j), \sigma_{i,j})$ with $i \leq j$ the indices of 2 vertices and $\sigma_{i,j}$ equal to 1 or 0 if the vertices i, j are connected or not. Recall there are $N(N-1)/2$ pairs of integers i, j in $\{1, \dots, N\}$ with $i < j$ the indices of 2 vertices and $\sigma_{i,j}$ equal to 1 or 0 if the vertices i, j are connected or not. It is obvious that any ordering of these edge vectors describe the same graph. Recall there are $N(N-1)/2$ pairs of integers i, j in $\{1, \dots, N\}$ with $i \leq j$. Consider the set of functions $f : \mathbf{R}^{N(N-1)/2 \times 2} \rightarrow \mathbf{R}^{N(N-1)/2 \times 2}$ that are equivariant to the permutations of columns then theorem 1 says the function f can be approximated with arbitrary accuracy by Transformers on edge input.

The naive node Transformer can be defined as a Transformer with positional encodings. This graph Transformer will take as input the identity matrix and as positional encodings the padded adjacency matrix. This can be viewed as a one-hot encoding of each node's neighbors. Consider the set of continuous functions $f : \mathbf{R}^{N \times N} \rightarrow \mathbf{R}^{N \times N}$, then theorem 2 says the function f can be approximated with arbitrary accuracy by Transformers on node inputs.

From these two observations on the universality of graph Transformers, we get as a corollary that these 2 types of Transformers can approximate solutions of the graph isomorphism problem. In each case, pick a function that is invariant under node index permutations and maps non-isomorphic graphs to different values and apply theorem 1 or 2 that shows there is a Transformer approximating that function to an arbitrarily small error in the L^p distance. This is an interesting fact since it is known that the discrimination power of most message passing graph networks is upper bounded by the Weisfeiler-Lehman test which is unable to distinguish some graphs.

This may seem strange since it is unlikely there is an algorithm solving the graph isomorphism problem in polynomial time to the number of nodes N , and we address this issue in the notes below.

Note 1: Only an approximate solution. The universality theorems do not state that Transformers solve the isomorphism problem, but that they can approximate a solution. They only learn the invariant functions only up to some error so they still can mislabel graphs.

Note 2: Estimate of number of Transformer blocks. For the approximation of the function f by a Transformer to be precise, a large number of Transformer blocks will be needed. In [40], it is stated that the universal class of function is obtained by composing Transformer blocks with 2 heads of size 1 followed by a feed-forward layer with 4 hidden nodes. In [41] section 4.1, an estimate of the number of blocks is given. If $f : \mathbf{R}^{d \times n} \rightarrow \mathbf{R}^{d \times n}$ is L -Lipschitz, then $\|f(\mathbf{X}) - f(\mathbf{Y})\| < \varepsilon/2$ when $\|\mathbf{X} - \mathbf{Y}\| < \varepsilon/2L = \delta$. In the notation of [41], the LPE has constants $p = 2$ and $s = 1$. If g is a composition of Transformer blocks then an error $\|f - g\|_{L^p} < \varepsilon$ can be achieved with a number of Transformer blocks larger than

$$\left(\frac{dn}{\delta}\right) + \left(\frac{p(n-1)}{\delta^d} + s\right) + \left(\frac{n}{\delta^{dn}}\right) = \frac{dn2L}{\varepsilon} + \frac{2(n-1)(2L)^d}{\varepsilon^d} + 1 + \frac{n(2L)^{dn}}{\varepsilon^{dn}}$$

In the case of the node encoder described above, $n = d = N$ (the number of nodes) and the last term in the sum above becomes $N(2L/\varepsilon)^{N^2}$, so the number of parameters and therefore the computational time is exponential in the number of nodes for a fixed error ε . Note that this bound on the number of Transformer blocks might not be tight and might be much lower for a specific problem.

Note 3: Learning invariance to label permutations. In the above proof, the Transformer is assumed to be able to label all isomorphic graphs into the same class within a small error. However, given a graph of N nodes, there are $N!$ different node labeling permutations, and they all need to be mapped to the same output class. It seems unlikely that such function can be learned with polynomial complexity to N .

Following these observations, it does not seem appropriate to compare Transformers to the WL test as is the custom for graph neural networks and we think at this point we should seek a new measure of expressiveness of graph Transformers.

C.3 Expressivity of the node-LPE

Here, we want to show that the proposed node-LPE can generate a unique node identifier that allows our Transformer model to be a universal approximator on graphs, thus allowing us to approximate a solution to graph isomorphism.

Recall the node LPE takes as input an $N \times m \times 2$ tensor with m the number of eigenvalues and eigenvectors that are used to represent the nodes. The output is a $N \times k$ tensor. Notice that 2 non-isomorphic graphs on N nodes can have the same $m < N$ eigenvalues and eigenspaces and disagree on the last $N - m$ eigenvalues and eigenspaces. Any learning algorithm missing the last $N - m$ pieces of information won't be able to distinguish these graphs. Here we will fix some m and show that the resulting Transformer can approximately classify all graphs with $N \leq m$.

Fix some linear injection $M : \mathbf{R}^{N \times 2 \times m} \rightarrow \mathbf{R}^{N \times k \times m}$. Let G be a graph and $U \subset \mathbf{R}^{N \times 2 \times m}$ be bounded set containing all the tensor representations of graphs T_G and let R be the radius of a ball containing $M(U)$. Consider the set $C^0(\mathbf{B}_R^{N \times k \times m}, \mathbf{R}^{N \times k \times m})$ of continuous functions of the closed radius R ball in $\mathbf{R}^{N \times k \times m}$. Finally, denote by $S : \mathbf{R}^{N \times k \times m} \rightarrow \mathbf{R}^{N \times k}$ the linear function taking the sum of all values in the m dimension. The following universality result for LPE Transformers is a direct consequence of theorem 2.

Proposition 1. *For any $1 \leq p < \infty$, $\varepsilon > 0$ and any continuous function $F : \mathbf{B}_R^{N \times k \times m} \rightarrow \mathbf{R}^{N \times k}$, there is an LPE Transformer g such that the L^p distance between $M \circ f \circ S$ and g is smaller than ε .*

As a corollary, we get the same kind of approximation to solutions of the graph isomorphism problem as with the naive Transformers. Let f be a function of $C^0(\mathbf{B}_R^{N \times k \times m}, \mathbf{R}^{N \times k \times m})$ that maps $M(T_G)$ to a value that is only dependent of the isomorphism class of the graph and assigns different values to different isomorphism classes. We can further assume that f takes values that are 0 for all but one coordinate in the k dimension. The same type of argument is possible for the edge-LPE from figure 8.

C.4 Comparison of the learning complexity of naive graph Transformers and LPE

We now argue that while the LPE Transformer and the naive graph Transformers of section C.2 can all approximate a function f solution of the graph isomorphism problem, the complexity of the learning problem of the LPE is much lower since the spaces it has to learn are simpler.

Naive Node Transformer. First recall that the naive node Transformer learns a map $f : \mathbf{R}^{N^2} \rightarrow \mathbf{R}^{N^2}$. In this situation, each graph is represented by $N!$ different matrices which all have to be identified by the Transformer. This encoding also does not provide any high-level structural information about the graph.

Naive Edge Transformer. The naive edge Transformer has the same difficulty since the function its learning is $\mathbf{R}^{N(N-1)} \rightarrow \mathbf{R}^{N(N-1)}$ and the representation of each edge depend on a choice of labeling of the vertices and the $N!$ possible labelings need to be identified again.

Node-LPE Transformer. In the absence of eigenvalues with multiplicity > 1 , the node LPE that learns a function $\mathbf{R}^{N \times 2 \times m} \rightarrow \mathbf{R}^{N \times k}$ does not take as input a representation of the graph that depends on the ordering of the nodes. It does, however, depend on the choice of the sign of each of the eigenvectors so there are still 2^N possible choices of graph representations that need to be identified by the Transformer but this is a big drop in complexity compared to the previous $N!$. The eigenfunctions also provide high-level structural information about the graph that can simplify the learning task of the graph.

Edge-LPE Transformer. Finally, the edge LPE of appendix A uses a graph representation as input that is also independent of the sign choice of the eigenvectors so each graph has a unique representation (considering the absence of eigenvalues with multiplicity > 1). Again, the eigenfunctions provide high-level structural information that is not available to the naive Transformer.

LPE Transformers for non-isospectral graphs. Isospectral graphs are graphs that have the same set of eigenvalues despite having different eigenvectors. Here, we argue that the proposed node LPE can approximate a solution to the graph isomorphism problem for all pairs of non-isospectral graphs, without having to learn invariance to the sign of their eigenvectors nor their multiplicities. By considering only the eigenvalues in the initial linear layer (assigning a weight of 0 to all ϕ), and knowing that the eigenvalues are provided as inputs, the model can effectively learn to replicate the input eigenvalues at its output, thus discriminating between all pairs of non-isospectral graphs. Hence, the problem of learning an invariant mapping to the sign of eigenvectors and multiplicities is limited only to non-isospectral graphs. Knowing that the ratio of isospectral graphs decreases as the number of nodes increases (and is believed to tend to 0) [35], this is especially important for large graphs and mitigates the problem of having to learn to identify 2^N with eigenvectors with different signs. In Figure 11, we present an example of non-isomorphic graphs that can be distinguished by their eigenvalues but not by the 1-WL test.

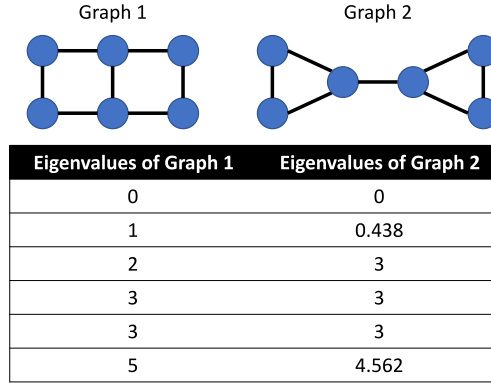


Figure 11: Example of non-isomorphic non-isospectral graphs that can be distinguished by the eigenvalues of their Laplacian matrix, but not by the 1-WL test.